

Improving function templates with Class Template Argument Deduction

This paper proposes

- Allowing Function Templates to consider Class Template Argument Deduction during Function Template Argument Deduction.
- Allowing deduction guides for Function Templates

This greatly improves Function Template Argument Deduction in important use cases as the following examples, which will be discussed in more detail below, illustrate:

C++17	Proposed
<pre>// Example 1: "Output" version of std::quoted // Needs many overloads since no CTAD for function template arguments template<class charT> T1 quoted(const charT* s, charT delim = charT('\"'), charT escape = charT('\\')); template<class charT, class traits, class Allocator> T12 quoted(const basic_string<charT, traits, Allocator>& s, charT delim = charT('\"'), charT escape = charT('\\')); template<class charT, class traits, class Allocator> T14 quoted(basic_string_view<charT, traits, Allocator>& s, charT delim = charT('\"'), charT escape = charT('\\'));</pre>	<pre>// Example 1: Allow CTAD to deduce function template argumer template<class charT, class traits, class Allocator> T1 quoted(basic_string_view<charT, traits, Allocator>& s, charT delim = charT('\"'), charT escape = charT('\\'));</pre>
<pre>// Example 2: Passing 'In' parameter by value or const & by efficiency template <typename T> void do_f(T t); // Implements f template <typename T> inline void f(T && t) { do_f<typename boost::call_traits<decay_t<T>>::type>(t); } f(7); // Pass by-value f(vector {1, 2, 3, 4}); // Passed by const &</pre>	<pre>// Example 2: Allow Deduction Guides for Function Templates template <typename T> void f(T t); // Implement here. No need template <typename T> f(T t) -> f<typename boost::call_traits f(7); // Pass by-value f(vector {1, 2, 3, 4}); // Passed by const &</pre>

Argument deduction for function templates

A signature feature of C++ that has greatly contributed to the success of the standard template library is that function templates generally deduce their template arguments, so algorithms can usually be called for any valid arguments and Function Template Argument Deduction chooses the right specialization. However, Function Template Argument Deduction fails to make many “expected” deductions because it does not consider Class Template Argument Deduction like other declarations do, as we now propose:

<pre>// Point class template along the lines of P0267R8 template<class T> Point { T x; T y; };</pre>	
C++17	Proposed
<pre>// Function Template Argument Deduction fails distance<double>({0.0, 0.0}, {3.0, 4.0});</pre>	<pre>// Fix by deducing argument from initializer distance({0.0, 0.0}, {3.0, 4.0});</pre>

Indeed, the standard library goes to great lengths to work around this by add many function template overloads that merely duplicate Class Template Argument Deduction. Consider the “Output” version of std::quoted:

C++17	Proposed
<pre>// Needs many overloads since no CTAD for function template arguments template<class charT> T11 quoted(const charT* s, charT delim = charT('\"'), charT escape = charT('\\')); template<class charT, class traits, class Allocator> T12 quoted(const basic_string<charT, traits, Allocator>& s, charT delim = charT('\"'), charT escape = charT('\\')); template<class charT, class traits, class Allocator> T14 quoted(basic_string_view<charT, traits, Allocator>& s, charT delim = charT('\"'), charT escape = charT('\\'));</pre>	<pre>// Example 1: CTAD deduces s argument as in ordinary declar template<class charT, class traits, class Allocator> T1 quoted(basic_string_view<charT, traits, Allocator>& s, charT delim = charT('\"'), charT escape = charT('\\'));</pre>

Many other standard library function templates, such as basic_string::append, basic_string::append(), basic_string::find(), regex_match(), etc. have to go through similar error-prone contortions to support Function Template Argument Deduction. Indeed, since we recommend that programmers working with text take string_views in their function templates, they will have to frequently create similar manual overloads themselves.

As just one more example, with the addition of optional, function templates that take optional arguments, will have to sacrifice natural Function Template Argument Deduction:

<pre>// Point class template along the lines of P0267R8 template<class T> void f(optional<T>);</pre>	
C++17	Proposed
<pre>f<int>(7); // Do I really need to do this?</pre>	<pre>f(7); // Proposed: No!</pre>

One technical point deserves mention here. It may be that the function template parameter is a more complex partial specialization of a class template than in the example above. For example,

```
template<typename T> void f(pair<int, T>);
```

In such a case, we deduce the pair<int, T> by following the same process as used for alias templates in [P1021R1](#), thereby deducing T. For example, f({}, 2L) will deduce f<long>, which takes a pair<int, long> as an argument. For the technical details, see P1021R1, which has a step-by-step walkthrough for deducing pair<int, T>.

Deduction guides for function templates

While Class Template Argument Deduction uses both implicit and explicit deduction guides, Function Template Argument Deduction in effect uses only implicit guides (ordinary Function Template Argument deduction). Given that it often proves helpful to override the implicit deduction behavior of class templates, would the same be true for function templates? The answer is a resounding “yes”.

std::reference_wrapper is commonly used to effect pass-by-reference to function templates with generic parameter types, but this is rarely used outside the standard library we believe due to the ugliness of forcing the correct deduction. If, as we propose, deduction guides could be provided for implicit guides, unwrapping would be straightforward, both inside the standard library and in user code:

C++17	Proposed
<pre>// Helper for converting reference wrappers to references template <typename T> struct unwrap { using type = T }; template <typename T> struct unwrap<std::reference_wrapper<T>> { using type = T & } template <typename T> using unwrap_t = typename unwrap<T>::type; template<typename T> void f_impl(T t); // We want f to take its argument by value or ref based // whether or not it is a reference_wrapper template <typename T> void f(T && t) { f_impl<unwrap_t<T>>(t); }</pre>	<pre>// Unwrap reference_wrapper with deduction guide template <typename T> void f(T t); template <typename T> f(reference_wrapper<T>) -> f<T &>;</pre>

Not only does the deduction guide make the code much simpler, it moves the unwrapping of the reference_wrapper from being hidden inside the body of f to being advertised in f's interface. In real life, this is often even hidden more deeply as many implementation of, say, for_each do not unwrap reference wrappers until they actually invoke the callable.

As another example, for function templates that accept an “in” argument, it is common to want to accept small argument types, such as int, by value and large or uncopyable types by const &. Unfortunately, this is rarely done as the implicitly-generated rules for Function Template Argument Deduction do not readily provide the desired behavior. With deduction guides, this again naturally simplifies to a form that no longer requires examining the body of f to understand how arguments are passed:

C++17	Proposed
<pre>template <typename T> void f_impl(T t); template <typename T> inline void f(T && t) { do_f<typename boost::call_traits<decay_t<T>>::type>(t); }</pre>	<pre>template <typename T> void f(T t); template <typename T> f(T t) -> f<typename boost::call_traits<T>::type>;</pre>

Note: Earlier versions of this paper proposed allowing deduction guides to optionally specify which of an overloaded function template they were guiding. We defer this both due to complexity and the absence of compelling use cases. This can always be added later if it proves worthwhile. Indeed, class template constructors also form an overload set, and such a feature has not proved necessary for Class Template Argument Deduction.